

Inventory Project Interview Questions

Likely interviewer questions, strong answers, demo script, and discussion points

Project	Production-Ready Containerized Inventory & Order Management System
Stack	React, FastAPI, PostgreSQL, Docker, Docker Compose, Git
Purpose	First-round software engineer technical assessment and interview preparation

How to use this document

Use this document as a practical checklist before submission and as talking points during the technical interview. It focuses on what is complete, what still needs attention, and what can be added after submission without creating unnecessary risk.

Interview Preparation: How to Explain the Project

Use the following explanation when the interviewer asks: "Tell me about your project."

This is a full-stack Inventory and Order Management System built with React, FastAPI, PostgreSQL, and Docker. It manages products, customers, orders, and inventory tracking. The backend enforces all important business rules such as unique product SKU, unique customer email, insufficient stock validation, backend-calculated order total, automatic stock deduction, and stock restoration on cancellation. The frontend provides dashboard, product management, customer management, order creation, status updates, and order details view. The complete system is containerized using Docker Compose and prepared for deployment on Render and Vercel.

Architecture and System Design

Question	Strong answer
Why did you choose FastAPI?	FastAPI is lightweight, fast, API-first, supports automatic OpenAPI documentation, Pydantic validation, dependency injection, and is simple to containerize. For this assessment, it fits better than a heavier framework because the core requirement is REST APIs.
Explain your system architecture.	The frontend is a React/Vite app. It calls FastAPI endpoints through an Axios wrapper. FastAPI talks to PostgreSQL through SQLAlchemy ORM. Docker Compose runs frontend, backend, and database as separate services. Environment variables control database URL, CORS, and secrets.
What are the main modules?	Auth, products, customers, orders, dashboard, database, models, schemas, and CRUD/business logic. The frontend has AuthPage, Dashboard, ProductManager, CustomerManager, OrderManager, Message, and Pagination components.
Why did you separate routers and CRUD logic?	Routers handle HTTP input/output and dependency injection. CRUD/service functions contain database and business logic. This separation makes code easier to test, maintain, and extend.
What is the data flow when creating an order?	React sends customer_id and order items to FastAPI. Backend validates customer and products, checks stock, calculates line totals and total amount, reduces product quantity, saves order and order items, commits transaction, and returns the order response.

Backend and FastAPI

Question	Strong answer
What is dependency injection in FastAPI?	It is a way to declare reusable dependencies such as database sessions and current authenticated user. For example, routes can depend on <code>get_db</code> and <code>require_active_user</code> .
How are request validations handled?	Pydantic schemas validate fields such as email format, price greater than zero, quantity non-negative, SKU length, and required fields before business logic runs.
How do you return proper HTTP status codes?	201 for created resources, 200 for successful reads/updates, 204 for delete, 400 for invalid business conditions like insufficient stock, 401 for invalid token, 403 for disabled/unauthorized account, 404 for not found, and 409 for duplicate SKU/email.
Why should total amount be calculated in the backend?	The frontend can be modified by users. Backend calculation prevents tampering and ensures price and stock logic remain trustworthy.
How do you handle duplicate SKU or email?	Database unique constraints enforce uniqueness. <code>IntegrityError</code> is caught and converted into HTTP 409 Conflict with a clear message.
What is the purpose of <code>/health</code> and <code>/health/db</code> ?	<code>/health</code> confirms the API process is running. <code>/health/db</code> confirms database connectivity, which is important for deployment verification and monitoring.

Database and SQLAlchemy

Question	Strong answer
What tables are used?	Users, products, customers, orders, and order_items. Orders reference customers. Order items reference orders and products.
Why use a separate order_items table?	One order can contain multiple products. A separate order_items table models the one-to-many relationship and stores quantity, unit price, and line total.
Why store unit_price in order_items?	Product price may change later. Storing unit_price preserves historical order price at the time of purchase.
How do you prevent negative product quantity?	Pydantic validates quantity as non-negative and the database has a check constraint. Order creation also checks stock before reducing quantity.
What is a transaction and why important here?	A transaction ensures all changes succeed or fail together. In order creation, order row, order items, and stock deduction must be committed atomically.
How can concurrent orders affect stock?	Two users could order the same product at the same time. The backend uses database locking/transaction logic to prevent overselling. In production, row-level locking and isolation are important.

Authentication and Security

Question	Strong answer
How does login work?	User submits email and password. Backend verifies password hash, creates JWT access token, and frontend stores it. Axios attaches Bearer token to future requests.
Why hash passwords?	Plain passwords must never be stored. Hashing stores a one-way representation, so even if DB leaks, passwords are not directly exposed.
What is JWT?	JWT is a signed token containing claims such as user ID and expiration. Backend verifies the signature and expiration before allowing protected API access.
Why use SECRET_KEY environment variable?	JWT signing secret must not be hardcoded. It should differ by environment and stay outside Git.
What is CORS?	CORS controls which frontend origins can call the backend from a browser. It must include the deployed frontend URL.
What security improvement would you add next?	Role-based access control, refresh tokens, rate limiting, stronger password policy, audit logs, and secure HTTP-only cookies for token storage.

Frontend and React

Question	Strong answer
How is the frontend structured?	AppShell controls navigation and selected tab. Each module is separated into Dashboard, ProductManager, CustomerManager, and OrderManager. API calls are centralized in api.js.
Why use a central Axios instance?	It avoids repeating base URL, headers, token handling, timeout, and error handling in every component.
How do you manage form validation?	Basic validation is done on frontend for user experience, while backend validation remains the final authority.
Why add order details modal?	The assessment requires viewing order details. The modal displays customer, status, total, date, notes, and line items in a clear UI.
How do you refresh dashboard badges?	After create/update/delete operations, onChange refreshes summary metrics such as low stock and pending orders.
How does pagination work?	Backend returns items, total, page, limit, and pages. Frontend passes page changes to API and renders pagination component.

Docker and Deployment

Question	Strong answer
Why Dockerize the project?	Docker ensures consistent runtime across local, review, and deployment environments. It also satisfies the mandatory assessment requirement.
What services run in Docker Compose?	Frontend, backend, and PostgreSQL database. PostgreSQL uses a named volume for persistence.
Why does db hostname work only inside Docker?	Docker Compose creates a private network where service names resolve as hostnames. Outside Docker, local backend must use localhost and mapped DB port.
What should be deployed where?	Backend on Render, frontend on Vercel, PostgreSQL on Render/managed DB, and backend Docker image on Docker Hub.
What environment variables are important?	DATABASE_URL, CORS_ORIGINS, SECRET_KEY, ENVIRONMENT, LOW_STOCK_THRESHOLD, and VITE_API_URL.
What is the frontend Docker approach?	Build React app, then serve static dist files through Nginx with SPA fallback.

Testing and Quality

Question	Strong answer
What tests did you write?	Tests cover product CRUD, SKU uniqueness, validation, customer CRUD, email uniqueness, order creation, stock reduction, insufficient stock, status change, cancellation stock restore, dashboard summary, and health endpoint.
Why override authentication in tests?	Most endpoints are protected. Tests bypass auth dependency to focus on business logic and API behavior without needing login setup for every test.
Why use SQLite for tests?	It is lightweight and fast for local test execution. For production-level confidence, a PostgreSQL test container can be added later.
What is your expected test result?	After adding the order details test, expected result is 30 tests passed.
What would you add to CI?	GitHub Actions to install dependencies, run pytest, run frontend build, and optionally build Docker image.

Scenario-Based Questions

Question	Strong answer
What happens if two users order the last item at the same time?	The backend should use database transaction and row-level locking to prevent both orders from reducing the same stock. This is a concurrency risk I would handle with with_for_update and proper transaction boundaries.
What happens if frontend sends wrong total_amount?	Backend ignores frontend total and calculates total from database prices and quantities.
What happens if product is deleted but existing order references it?	Database restrictions should prevent deleting products linked to existing orders, or soft delete can be used in future.
How do you debug deployment CORS errors?	Check browser console, backend CORS_ORIGINS, frontend VITE_API_URL, HTTPS URLs, and whether backend returns correct Access-Control-Allow-Origin header.
How would you scale this system?	Use managed PostgreSQL, connection pooling, multiple backend workers, caching for dashboard aggregates, pagination, indexes, and background jobs for reports/alerts.
What is one weakness in current system?	Migrations and role permissions can be improved. The current system is assessment-ready but production could benefit from Alembic, RBAC, audit logs, and refresh tokens.

Quick Demo Script for Interview

- Open live frontend URL and login with demo credentials.
- Show dashboard metrics: total products, customers, orders, low stock, revenue, pending orders.
- Create a new product with SKU, price, and quantity.
- Create a new customer with unique email.
- Create an order for that customer and product.
- Show stock quantity reduced automatically after order creation.
- Open View Details modal to show customer, items, line total, and backend-calculated total.
- Cancel the order and show stock restored.
- Open API docs and show products/orders endpoints.
- Mention tests and Docker Compose setup.

Questions You Should Ask the Interviewer

- Would your team prefer event-driven stock movement tracking for inventory changes?
- Do you use separate staging and production environments for such deployments?
- Do you prefer JWT in local storage, HTTP-only cookies, or an SSO/IAM flow?
- Would the next priority be reporting, RBAC, or purchase order management?